

1. What difference do you observe?

When the logging level is changed from ERROR to DEBUG, more messages are recorded in the log file.

- (i) With logging.ERROR: only ERROR and CRITICAL messages are recorded.
- (ii) With logging.DEBUG: DEBUG, INFO, WARNING, ERROR, and CRITICAL messages can be recorded.

So, the DEBUG log file contains much more detailed information about the program.

2. Why does changing the logging level affect the messages recorded in the log file?

The logging level acts as a filter. It determines the minimum level of messages that the logging system will record.

1. What is exception handling?

Exception handling is a way of detecting and managing errors that happen while a Python program is running. Instead of allowing an unexpected error to stop the program, we can catch the exception and respond with a suitable message or action.

2. Why should we use exception handling?

Exception handling prevents the application from crashing because of invalid or unexpected input. It allows the program to give the user a useful message, record the problem in a log, and continue or terminate safely.

3. What is the difference between `try` and `except`?

The `try` block contains code that might cause an exception. The `except` block runs when a matching exception occurs and handles that exception.

4. When is the `else` block executed?

The `else` block is executed only when the code in the `try` block finishes without raising an exception.

5. When is the `finally` block executed?

The `finally` block is executed whether an exception occurs or not. It is useful for actions that should always happen, such as cleanup or displaying a completion message.

6. What is logging?

Logging records information about events that happen while an application runs. It is useful for debugging, understanding application behavior, finding errors, and monitoring a program without relying only on screen output.

7. What is the difference between `print()` and logging?

`print()` is mainly intended to display information directly to the user. Logging is designed to record application events and errors at different levels, such as `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`. Logging can also write these records to a file.

8. What happens when the logging level is set to `ERROR`?

When the logging level is `ERROR`, messages at `ERROR` and `CRITICAL` levels are recorded. `DEBUG`, `INFO`, and `WARNING` messages are not recorded.

9. What happens if we do not handle `ValueError` when converting user input using `int()`?

If the user enters something that cannot be converted to an integer, Python raises a `ValueError`. If it is not handled, the program can terminate with an error message instead of giving the user an opportunity to enter valid input.

10. Why should we avoid using a broad exception handler such as `except: pass`?

It hides errors and gives no information about what went wrong. This makes debugging difficult and can allow the program to continue in an incorrect state. Specific exceptions should be caught and handled appropriately.

11. Why is logging useful in a production application?

Logging provides a history of important application events, warnings, and errors. Developers and system administrators can use logs to diagnose problems, monitor behavior, and investigate failures without needing to reproduce every issue interactively.

12. What is the purpose of the `finally` block?

The `finally` block contains code that should run regardless of whether an exception occurs. In this assignment it is used to indicate that processing has completed and to record that the block was executed.